

Lab 08 - Cast-Presence Detection and Detector Evaluation

Course: Create Short Video Film using AI (TGS-2020505925) | Version v12.0 | 27 September 2026 Topic 04: AI Video Generation, Animation and Video Analytics Outcome: ELO4 | In-class time: 60 minutes of scheduled practical time

Competency	Statement
Ability A5	Design and apply machine-learning based methods for object detection, object tracking and activity recognition
Knowledge K8	Deep learning concepts
Knowledge K9	Object segmentation, detection and recognition

Scenario

A generated shot can quietly drop a character or add an extra. Before a detector is allowed to approve shots automatically, it must be evaluated. You run two built-in machine-learning detectors (a Viola-Jones face cascade and a HOG + linear-SVM people detector) on twelve storyboard frames with exact truth, implement IoU matching, sweep their operating parameters, and then use the face detector to check each frame's cast against the shot list.

What you will produce

An evaluation table (TP, FP, FN, precision, recall, F1 at three IoU thresholds), a parameter sweep with a chosen operating point, a cast-check table separating CAST DRIFT from DETECTOR ERROR, and a positive-control run on an AI-generated still.

Tools: Python 3 | opencv-python | NumPy | no weights download, no network

Environment

Install these once, before class if you can - the lab itself needs no network access after the dependencies are present:

```
python3 -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install "opencv-python>=4.10" numpy
python3 -c "import cv2, numpy; print(cv2.__version__, numpy.__version__)"
```

Required: opencv-python>=4.10, numpy.

API currency. Everything taught here runs on the base opencv-python wheel plus NumPy - no contrib modules and no downloaded model weights. SSIM is implemented in NumPy because cv2.quality is contrib-only. Generative steps are optional: every lab supplies SIMULATED reference media so it can be completed offline, and asks you to record the model ID and date if you generate your own.

Workflow

1. Self-test IoU
2. Run the face and person detectors

3. Match predictions to truth at three IoU thresholds
4. Sweep parameters and pick an operating point
5. Check each frame's cast against the shot list

Files in this folder

Path	What it is
reference/frames/frame_00.png ... frame_11.png	Twelve schematic storyboard frames with exact truth (SIMULATED)
reference/positive-control/retail-adults-reference.png	AI-GENERATED still of two fictional adults, with PROVENANCE.md
data/eval-config.json	IoU thresholds, parameter sweeps, matching rule and the operating-point rule
detect_eval.py	Runnable script - see the steps below
PROMPTS.md / PROMPTS.pdf	The reusable prompt and specification pack
out/	Everything you produce (created on first run)

Provenance. Every image and clip in reference/ is either a SIMULATED classroom illustration of *The Last Ferry*, rendered deterministically with OpenCV and labelled on the frame, or an AI-generated reference copied unchanged with a PROVENANCE.md beside it. Nothing in this folder may be relabelled: a simulated stand-in is never presented as model output or as a photograph.

Steps

1. Read data/eval-config.json: the three IoU thresholds, the minNeighbors and SVM weight sweeps, the one-to-one matching rule and the operating-point rule.
2. Confirm both detectors ship inside opencv-python - no download.

```
python3 -c "import cv2,os;p=cv2.data.haarcascades+'haarcascade_frontalface_default.xml';
print(os.path.exists(p), len(cv2.HOGDescriptor_getDefaultPeopleDetector()))"
```

3. Compute IoU by hand for [0, 0, 100, 100] and [50, 0, 100, 100]: intersection 5000, union 15000, IoU = 1/3. The script self-tests the same case before it runs.
4. Run the evaluation with the cast check.

```
python3 detect_eval.py --frames reference/frames --truth data/ground-truth.json --config
data/eval-config.json --cast data/shot-cast.json --out out
```

5. Read the evaluation lines. The HOG person detector scores well at IoU 0.3 and 0.5 and collapses at 0.7: its boxes are the right objects with loose edges. Explain why the IoU threshold is part of the metric, not a detail.
6. Open out/pr-sweep.csv. Apply the operating-point rule: maximise recall subject to precision ≥ 0.60 . Record the minNeighbors and SVM weight you choose.
7. Read the CAST CHECK. frame_04 and frame_09 disagree with the shot list and with the detector's truth count: CAST DRIFT - the frame is wrong. frame_06 disagrees only because the detector fired twice: DETECTOR ERROR.
8. Open out/overlay_frame_00.png (green = truth, blue = face, red = person) and check one frame by eye.
9. Run the positive control on the AI-generated still.

```
python3 detect_eval.py --frames reference/positive-control --truth data/positive-control-truth.json --config data/eval-config.json --out out/positive-control
```

10. Compare: the detectors that were near-perfect on schematic frames produce many false positives on a realistic image. Write two sentences on domain shift and why one image proves nothing either way.
11. Decide in out/detector-decision.md: may the face detector auto-approve shots, or only route disagreements to a reviewer? Use your numbers.
12. Explain in one paragraph how a segmentation mask would differ from a detection box for isolating MEI to regrade her raincoat.

Verify

The IoU self-test passes; the evaluation prints nine detector rows; the cast check flags frame_04 and frame_09 as CAST DRIFT and frame_06 as DETECTOR ERROR.

Expected outputs

- out/detections.json, out/evaluation.csv, out/pr-sweep.csv
- out/cast-check.csv and out/overlay_frame_00.png
- out/positive-control/evaluation.csv
- out/detector-decision.md

Evidence checklist

Submit all of the following. Each item is something an assessor can look at.

- ☐ Hand IoU calculation matching the self-test
- ☐ Evaluation table at three IoU thresholds with your explanation of the 0.7 collapse
- ☐ Chosen operating point with the precision constraint shown
- ☐ Cast-check table distinguishing CAST DRIFT from DETECTOR ERROR
- ☐ Positive-control result and the auto-approve decision

Troubleshooting

Symptom	What to do
'cascade failed to load'	Reinstall opencv-python; cv2.data.harcascades points inside the package.
'frame files and truth frame keys must match'	Every image in --frames needs an entry in the truth file, even an empty list.
The positive-control run shows many false positives	Expected: that is the domain-shift lesson. Do not tune on one image.
Different numbers on another machine	Detector output can vary slightly by OpenCV version; record yours.

References used in this lab

- [S13] M. Tschochohei and F. Schenker, 'Multimodal Generative AI in the Enterprise: From Pixels to Profit', Packt, June 2026, ISBN 978-1-80611-167-1 - *Ch.5 pp.56-75; Ch.7 pp.106-119; Ch.8 pp.127-143*.

VAE, GAN and diffusion mechanisms and failure modes; prompt structure and subject lock; binary masks (white = editable), inpainting, outpainting and mask-free editing; spatial versus temporal coherence, 3-D U-Nets and diffusion transformers over spatiotemporal patches; long-running operation polling; responsible-AI risks (voice cloning, deepfakes, likeness) and transparency (C2PA Content Credentials, invisible watermarking, model cards).

- [S14] A. Mewada, M. A. Ansari, S. Ahmad and N. Singh (eds), 'AI-Generated Image and Video Synthesis', IEEE Press / Wiley, 2026, ISBN 9781394403110 - *Ch.1 pp.1-11; Ch.3 pp.35-44; Ch.7 pp.94-100*.

Frame-based versus temporal video generation and the temporal-consistency challenge; the sensors-to-synthesis layered architecture and split computing between edge and cloud; evaluation (FID, KID, LPIPS, CLIP faithfulness, FVD plus optical-flow agreement); memory = parameters x precision, quantisation to b bits scales by b/32, diffusion latency proportional to S steps (S x F for video).

- [S17] OpenCV 4.13 Python package and FFmpeg, verified locally on the delivery image - *cv2 4.13.0 (opencv-python 4.13.0.92); ffmpeg/ffprobe optional*.

Every lab command runs on the base opencv-python wheel plus NumPy: VideoCapture / VideoWriter, cvtColor, calcHist/compareHist, Canny, ORB_create + BFMatcher, HuMoments, inpaint, CLAHE, filter2D, fastNIMeansDenoisingColored, Haar cascades from cv2.data.harcascades, HOGDescriptor_getDefaultPeopleDetector, CamShift and calcOpticalFlowFarneback. SSIM is implemented in NumPy because cv2.quality is contrib-only. FFmpeg is used only when present, to mux audio into the final export.

© 2026 Tertiary Infotech Academy Pte Ltd. UEN: 201200696W. Course material for TGS-2020505925, version v12.0.